

Relatório de Auditoria de Segurança

Guilda

Auditoria: 28 de agosto de 2026 · Revisão: 29 de agosto de 2026 · branch `design-system`

Esta revisão registra a correção dos achados e um teste de intrusão ativo. TODOS os 15 achados foram endereçados: 14 corrigidos e verificados e 1 mitigado (o resíduo de farm de XP dentro da Contabilidade, aceito conscientemente). Não há mais pendências — as baixas e informativas, antes recomendação, também foram implementadas e verificadas nesta rodada.

ESCOPO AUDITADO

- Todo o código-fonte em `src/` (150 arquivos .ts/.tsx): 20 arquivos de Server Actions com 86 actions exportadas, 2 route handlers, todas as páginas do App Router e os módulos de domínio/lib.
- 62 migrations SQL em `src/db/migrations/` (RLS, políticas, privilégios e funções SECURITY DEFINER).
- Arquivos de implantação: `Dockerfile`, `docker-compose.yml`, `docker-compose.dev.yml`, `docker/prod/`, `docker/dev/`, `next.config.ts`, `.dockerignore`, `.gitignore`.
- Scripts operacionais em `scripts/` (seed, importação, worker do Telegram, verificação de RLS, E2E).
- Histórico do Git (busca por segredos commitados) e os arquivos `.env.example` / `.env.production.example`.

STACK DETECTADA

Linguagem / framework	TypeScript estrito sobre Next.js 16.2.11 (App Router, Server Actions, <code>proxy.ts</code> no lugar do middleware).
ORM / acesso a dados	Drizzle ORM 0.45 com driver <code>node-postgres</code> sobre PostgreSQL 17.
Autenticação	better-auth 1.6.23 com o plugin <code>organization</code> (multi-tenant), sessão por cookie, rate limiting em banco.
Frontend	React 19 + Tailwind CSS 4 + shadcn/ui (Radix). Renderização em Server Components; nenhuma API REST pública além de <code>/api/auth/*</code> e <code>/api/telegram/webhook</code> .
Implantação	Docker multi-estágio (output standalone do Next) + Postgres + Caddy no Compose; migrations aplicadas no boot por <code>scripts/start-production.mjs</code> .
Integrações	Bot do Telegram (polling ou webhook) e Anthropic Claude para classificar informativos; BrasilAPI para consulta de CNPJ.

NOTA METODOLÓGICA — COMO CADA CATEGORIA FOI MAPEADA PARA ESTA STACK

1. Banco sem tranca

Não é Supabase. O mecanismo de isolamento desta stack tem duas camadas, e as duas foram auditadas: (a) toda leitura/escrita de domínio passa por `withOrgTx(orgId, ...)`, que abre transação e executa `SET LOCAL app.org_id`; (b) Row Level Security no Postgres com a política `org_isolation` usando `current_setting('app.org_id', true)`, valendo porque a aplicação conecta com o role dedicado não-superuser `guilda_app`. Foram conferidas as 42 tabelas de domínio contra as migrations de RLS, coluna a coluna, além das tabelas do better-auth (que não têm RLS e dependem de filtro manual por `organization_id`).

2. Permissão definida no navegador

O equivalente a “endpoint privilegiado” aqui é a Server Action. Cada uma das 86 actions exportadas foi cruzada com o gate de papel que a interface aplica na tela correspondente (`canDistributeClanTasks`, `canManageFiscalPortfolio`, `canManageClanCommitments`, `canManageClanMembership`, `canHandleInformatives`, `canViewClan`, `parseClanTab`), verificando se o servidor repete a decisão. Server Actions são endereçáveis por qualquer usuário autenticado que extraia o `Next-Action` id dos chunks estáticos públicos — a navegação da UI não é fronteira de segurança.

3. IDOR

Percorrido todo handler que recebe um identificador por parâmetro de rota, query ou corpo — as 86 Server Actions, os 2 route handlers e os carregadores de página que leem `params` / `searchParams` — checando se a consulta filtra por `org_id` além do id do objeto. Nenhuma amostragem: a varredura foi exaustiva por arquivo.

4. Chaves expostas

Busca por literais de segredo em `src/`, `scripts/`, `docker/`, `docs/`, `README.md`, `Dockerfile`, arquivos `Compose` e exemplos de `.env`; conferência dos defaults de expansão de variável (`${VAR:-...}` versus `${VAR:?...}`) no `Compose`; varredura do histórico do `Git` por `.env`, chaves privadas e tokens; e conferência do que sai no bundle do frontend (variáveis `NEXT_PUBLIC_*`).

5. Inputs sem tratamento (XSS)

Busca dos sinks do `React/Next` em todo `src/`: `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function`, `srcDoc`, `javascript:` e todo `href={...}` dinâmico. No servidor, verificado se texto de usuário entra em `HTML` de mensagens do `Telegram` (`parse_mode`) ou em qualquer template.

Todos os achados deste relatório foram verificados no código-fonte real, com arquivo e linha. Nada aqui é especulativo: onde uma categoria não produziu achado, isso está dito explicitamente.

Resumo executivo

A auditoria percorreu **86 Server Actions**, **2 route handlers**, todas as páginas do App Router, **62 migrations** e os arquivos de implantação. Somando a 2ª rodada de varredura ativa, foram **15 achados**, nenhum crítico — e **todos os 15 foram endereçados**: 14 corrigidos e verificados, 1 mitigado com resíduo aceito. Um teste de intrusão ativo, como invasor de menor privilégio, não abriu nenhuma brecha adicional.



Estado da remediação



Todos os 15 achados foram endereçados — das duas altas às informativas. O único não “corrigido” é o F2, **mitigado**: o gate foi implementado e o resíduo de farm de XP dentro da Contabilidade foi aceito conscientemente.



CATEGORIA	ACHADOS	VEREDITO
3. IDOR e roteamento por parâmetro	2	Nenhum IDOR de objeto-por-id: os 86 handlers resolvem o objeto com <code>and(eq(tabela.id, ...), eq(tabela.orgId, ctx.orgId))</code> dentro de <code>withOrgTx</code> , e o pentest cross-tenant confirmou 0 leituras e 0 escritas de outro tenant. Aqui entram dois achados de parâmetro de rota: o F6 (matcher do proxy, cosmético) e o F14, um open redirect no <code>?next=</code> do sign-in contornável por barra invertida — corrigido nesta revisão.
4. Chaves expostas (hardcode)	5	Nenhuma chave real de produção no código, nos configs ou no histórico do Git — só credenciais de demonstração/desenvolvimento. Na remediação ganharam trava: <code>env-guard</code> recusa segredos fracos/placeholder no boot de produção (F9/F10), o seed aborta fora do ambiente local (F7), a porta do Postgres de dev ficou em loopback (F8) e a planilha com dados de cliente foi apagada e barrada no <code>.gitignore</code> (F11).
5. Inputs sem tratamento (XSS)	2	Nenhum sink de XSS no projeto — nem no cliente nem no servidor. Os dois pontos de endurecimento foram fechados: a CSP passou a usar nonce + <code>'strict-dynamic'</code> em produção, sem <code>'unsafe-inline'</code> (F12, verificado no navegador), e a armadilha do escape sem consumidor do Telegram foi removida (F13).

Pontos fortes — o que está protegido

Cada item abaixo foi verificado no código, não presumido. Esta seção também serve de prova de cobertura da auditoria.

1. Isolamento de inquilino em duas camadas, aplicado sem exceção

`src/db/org-tx.ts:19-30` abre transação e executa `SET LOCAL app.org_id` antes de qualquer query; a política `org_isolation` compara `org_id` com `current_setting('app.org_id', true)`. A aplicação conecta com o role dedicado não-superuser `guilda_app` (`src/db/index.ts:6-15`, `docker/prod/init/01-roles.sh:7`), que é o que faz o RLS valer. Não foi encontrada nenhuma query de domínio fora de `withOrgTx`: os únicos usos diretos de `db.*` (`layout.tsx:16`, `action-context.ts:35`, `auth.ts:55`, `telegram/actions.ts:42`, `telegram/handlers.ts:55`, `telegram/worker.ts:307`) tocam apenas tabelas do better-auth e todos filtram por `organization_id`.

2. Zero IDOR em 86 Server Actions e 2 route handlers

Varredura exaustiva, arquivo por arquivo. Todo objeto lido por id usa `and(eq(tabela.id, ...), eq(tabela.orgId, ctx.orgId))`, e toda mutação repete o filtro de organização no `WHERE` mesmo já estando dentro do RLS. Os poucos `where(eq(...))` de condição única (`campaigns/templates/actions.ts:306,310`, `company-flow-actions.ts:400,514`, `informatives/confirm.ts:283`) atualizam linhas que um `SELECT ... FOR UPDATE` escopado por organização já havia resolvido na mesma transação.

3. Autorização por fatos lidos do banco, nunca pela interface

`src/lib/clans/facts.ts:21-56` carrega papel e liderança do banco e entrega a funções puras testadas em `src/domain/guild-permissions.ts`. `company-flow-actions.ts` protege 10 de 10 actions com `requireCorporateFlowClan`; `settings/clan-actions.ts` protege 8 de 8 com `requireClanManager`; `commitment-actions.ts`, 8 de 8 com `requireDistributionManager`; `fiscal-installment-actions.ts`, 6 de 6; `fiscal-actions.ts` e `office-fee-actions.ts`, todas. O Telegram repete exatamente o mesmo gate do painel (`telegram/ai-informative.ts:121-127`).

4. Ledger de XP imutável e idempotente no nível do banco

`REVOKE UPDATE, DELETE ON xp_ledger FROM guilda_app (0004_rls-xp-ledger.sql:10)` — estorno é lançamento negativo novo (`tasks/actions.ts:1019-1034`), nunca edição. O `xp_value` é congelado na criação e o comentário em `tasks/actions.ts:802` registra que nunca vem do cliente. Índices únicos parciais impedem crédito duplo por evento e por ano de fechamento (`0015_thankful_runaways.sql:3`).

5. Máquina de estados de missão validada no servidor, com lock de linha

`tasks/actions.ts:288-496` trava a linha com `.for("update")`, checka a intenção declarada da action (`allowedFrom`), roda `authorizeTransition` do domínio e executa o efeito de XP dentro da mesma transação. Transições concorrentes serializam no lock e a segunda falha na validação de estado.

6. Credencial Gov.br cifrada com AES-256-GCM e revelação restrita

`src/lib/company-flows/secrets.ts:22-48` usa IV aleatório de 12 bytes por registro e verifica o `authTag` na leitura. `revealCompanyFlowGovPassword` (`company-flow-actions.ts:583-608`) exige clã Societário ativo e `canReturnCompanyFlow` — responsável pelo fluxo, liderança ou admin/owner. Sem a chave, a aplicação recusa gravar em vez de guardar em claro (`:295-298`).

7. Webhook do Telegram com comparação em tempo constante e limites de corpo

`src/app/api/telegram/webhook/route.ts:14-22` usa `timingSafeEqual` com checagem prévia de tamanho; `:38-51` rejeita corpos acima de 1 MB tanto pelo `content-length` declarado quanto pelo tamanho real; `:53-60` valida o formato antes de despachar. O segredo é derivado por SHA-256 de `BETTER_AUTH_SECRET` + token do bot quando não há override (`telegram/config.ts:37-43`).

8. Funções SECURITY DEFINER mínimas e com privilégio revogado do PUBLIC

`0017_mute_red_wolf.sql:119-134` e `0018_faithful_spot.sql:5-48` definem `SET search_path`, retornam somente os identificadores necessários antes de o tenant ser conhecido, e fazem `REVOKE ALL ... FROM PUBLIC` seguido de `GRANT EXECUTE ... TO guilda_app`. Os tokens de vínculo do Telegram são armazenados apenas como digest SHA-256 (`telegram/link-token.ts:13-16`), com TTL de 10 minutos e uso único.

9. Nenhum sink de XSS em todo o frontend

Busca em todo `src/` por `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function`, `srcDoc` e `javascript:` — zero ocorrências. Não há renderizador de Markdown/HTML. Todos os `href={...}` são literais, template strings com ids do servidor, ou vêm de uma lista branca do servidor: os atalhos do dashboard são resolvidos por `resolveDashboardShortcuts` (`lib/dashboard-shortcuts.ts:112-121`), que descarta qualquer `target` fora das opções permitidas, e a action que os salva revalida contra os clãs visíveis (`dashboard/actions.ts:71-76`).

10. Gestão de segredos correta no Compose, no Git e no Docker

`docker-compose.yml:13,15,44,45,46` usa `${VAR:?mensagem}` para todo segredo obrigatório — nenhum `${VAR:-default}` em variável sensível. `.gitignore:33-35` ignora `.env*` com exceção só dos exemplos; `.dockerignore:4-5` repete a regra. A varredura do histórico do Git não encontrou `.env`, chave privada, token de bot nem chave de API em nenhum commit. Nenhuma variável `NEXT_PUBLIC_*` carrega segredo — só a URL pública da aplicação.

11. Validação Zod e verificação de sessão em 100% das Server Actions

As 86 actions começam por `requireMemberContext()` (ou um gate que o encapsula) e validam o input com `safeParse` antes de tocar o banco. Nenhuma aceita XP, nível ou status vindos do cliente.

12. Rate limiting em auth e cabeçalhos de segurança configurados

`src/lib/auth.ts:36-47` aplica 5 tentativas/min em `/sign-in/email`, 3/min em `/sign-up/email` e 10/min nos endpoints de convite, com armazenamento em banco (tabela `rate_limit`, `0005_sticky_speed_demon.sql:1`). `next.config.ts:22-36` define HSTS com `preload`, `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin` e `Permissions-Policy` restritiva.

Pontos fracos — os riscos centrais

Estes foram os riscos centrais no momento da auditoria (redigidos no presente para preservar o diagnóstico original). Todos os de severidade alta e média já foram **corrigidos e verificados** nesta revisão — o estado de cada um está no cartão do achado e na seção de validação ofensiva.

- 1. Uma família de Server Actions escapou do padrão de autorização da casa.** O projeto acerta a autorização em 79 das 86 actions com um padrão consistente e testado; as 7 de Fechamentos ficaram para trás e hoje aceitam qualquer membro da organização. Não é uma falha de arquitetura — é uma lacuna pontual num padrão que já existe pronto ao lado, em `commitment-actions.ts`.
- 2. O gate de gamificação mais valioso está exatamente na action sem gate.** `setYearClosed` é o único caminho do produto em que o próprio chamador se credita XP, e é justamente ele que não verifica papel. A combinação transforma um erro de autorização em destruição do ranking, que é o mecanismo central do produto.
- 3. A navegação da interface está sendo usada como fronteira de segurança em três pontos.** A aba de Fechamentos restrita ao clã Contabilidade (F1/F2), a flag `mustChangePassword` desligada pelo navegador (F3) e o matcher do proxy (F6) partem todos da mesma premissa equivocada: a de que aquilo que a UI não mostra, o servidor não precisa checar. Server Actions e endpoints do better-auth são endereçáveis diretamente.
- 4. A segunda camada de defesa do banco tem cobertura desigual e o verificador não vê a diferença.** 30 tabelas com `FORCE`, 12 sem — e as 12 sem são as do núcleo. O `check:rls` que existia para pegar essa regressão não olha para elas.
- 5. Credenciais de demonstração viajam junto com a imagem de produção.** O seed com senha pública criando conta `owner` está dentro do contêiner publicado, com `tsx` disponível e sem nenhuma trava de ambiente.

Achados — visão consolidada

SEVERIDADE	ESTADO	ARQUIVO:LINHA	DESCRIÇÃO
ALTA	CORRIGIDO	src/app/(app)/clans/[id]/closing-actions.ts:88 (createClosing) +5 localis	F1 Sete Server Actions de Fechamentos sem verificação de clã ou papel [cat. 2]
ALTA	MITIGADO	src/app/(app)/clans/[id]/closing-actions.ts:316-397	F2 setYearClosed credita XP para quem chamou, sem gate — farm de XP em massa [cat. 2]
MÉDIA	CORRIGIDO	src/db/migrations/0002_rls-domain.sql:11 (tasks), :15 (task_events) +5 localis	F4 12 tabelas de domínio têm RLS habilitado mas não forçado (FORCE) [cat. 1]
MÉDIA	CORRIGIDO	src/lib/clans/bootstrap.ts:236 (cleanup do offboarding) +2 localis	F15 Notificações do Telegram sobreviviam à saída do integrante da organização [cat. 1]
MÉDIA	CORRIGIDO	src/lib/auth.ts:25-33 +2 localis	F3 A troca obrigatória de senha temporária é desativada pelo próprio navegador [cat. 2]
MÉDIA	CORRIGIDO	src/app/(auth)/sign-in/page.tsx:14 (versão anterior)	F14 Filtro de open redirect do sign-in contornável por barra invertida e caracteres de controle [cat. 3]
MÉDIA	CORRIGIDO	scripts/seed.ts:7 +5 localis	F7 Seed cria uma conta owner com senha fixa, sem trava de ambiente, e vai para a imagem de produção [cat. 4]
BAIXA	CORRIGIDO	docker/dev/init/01-roles.sql:5 +4 localis	F8 Senha do role da aplicação fixa no init do Postgres de desenvolvimento [cat. 4]
BAIXA	CORRIGIDO	Dockerfile:16-18 +1 local	F9 Placeholder literal de BETTER_AUTH_SECRET no Dockerfile, sem validação própria de inicialização [cat. 4]
BAIXA	CORRIGIDO	next.config.ts:8-20	F12 CSP depende de 'unsafe-inline' em script-src [cat. 5]
INFORMATIVA	CORRIGIDO	scripts/check-rls.mjs:13-38 +1 local	F5 O verificador de RLS não cobre as tabelas do núcleo [cat. 1]
INFORMATIVA	CORRIGIDO	src/proxy.ts:30-43	F6 O matcher do proxy não acompanha as rotas protegidas do app [cat. 3]
INFORMATIVA	CORRIGIDO	src/lib/company-flows/secrets.ts:13-20	F10 FLOW_SECRETS_KEY aceita qualquer valor que decodifique para 32 bytes [cat. 4]
INFORMATIVA	CORRIGIDO	.gitignore +1 local	F11 Planilha com dados do escritório na raiz do repositório, sem regra no .gitignore [cat. 4]

SEVERIDADE	ESTADO	ARQUIVO:LINHA	DESCRIÇÃO
INFORMATIVA	CORRIGIDO	<code>src/lib/telegram/format.ts:3-9</code> +2 localis	F13 Função de escape do Telegram sem consumidor — armadilha latente [cat. 5]

Categoria 1 Banco sem tranca — isolamento de inquilino

O isolamento é a parte mais bem construída do projeto: RLS real, role dedicado não-superuser, `with0rgTx` em toda query de domínio e filtro explícito por `org_id` além da política — e o pentest confirmou que nenhuma leitura ou escrita atravessa o tenant. O achado F4 é de profundidade (12 tabelas sem `FORCE`, já corrigido); a 2ª rodada acrescentou o F15, um vazamento de fronteira pela ENTREGA — notificações do Telegram que sobreviviam à saída do integrante —, também corrigido.

MÉDIA CORRIGIDO F4 12 tabelas de domínio têm RLS habilitado mas não forçado (FORCE)

```
src/db/migrations/0002_rls-domain.sql:11 (tasks), :15 (task_events)
src/db/migrations/0004_rls-xp-ledger.sql:3 (xp_ledger)
src/db/migrations/0007_rls-clients.sql:4 (clients)
src/db/migrations/0009_rls-mission-templates.sql (mission_templates, mission_template_items)
src/db/migrations/0011_rls-accounting-closings.sql:2 (accounting_closings, accounting_closing_years)
src/db/migrations/0017_mute_red_wolf.sql (telegram_connections, telegram_link_tokens, telegram_preferences, telegram_outbox)
```

```
-- 0002_rls-domain.sql:11-17 - ENABLE, sem FORCE
ALTER TABLE "tasks" ENABLE ROW LEVEL SECURITY;
CREATE POLICY "org_isolation" ON "tasks"
  USING ("org_id" = current_setting('app.org_id', true));

ALTER TABLE "task_events" ENABLE ROW LEVEL SECURITY;
CREATE POLICY "org_isolation" ON "task_events"
  USING ("org_id" = current_setting('app.org_id', true));

-- compare com 0025_force-rls-informatives.sql:7, o padrão adotado depois:
ALTER TABLE "informatives" FORCE ROW LEVEL SECURITY;
```

POR QUE É EXPLORÁVEL

Das 42 tabelas de domínio, 30 recebem `FORCE ROW LEVEL SECURITY` e 12 ficaram só com `ENABLE`. Sem `FORCE`, o dono da tabela ignora a política — o RLS dessas 12 tabelas só vale enquanto a aplicação conectar com um role que não seja o `postgres`. Hoje conecta (`src/db/index.ts:6-15`, `docker-compose.yml:43`), então não há furo em operação normal. O problema é que a própria base de código antecipa o cenário contrário: `scripts/start-production.mjs:22-28` avisa em runtime que, se `MIGRATION_DATABASE_URL` estiver ausente, as migrations rodam com `DATABASE_URL` — e o `docker-compose.yml:5-6` documenta o deploy com Postgres externo, em que apontar as duas variáveis para o mesmo role é o erro natural. Nessa configuração, o isolamento continuaria valendo nas 30 tabelas com `FORCE` e desapareceria silenciosamente em `tasks`, `xp_ledger`, `clients` e `accounting_closings` — justamente as do núcleo. As migrations 0025, 0027 e 0032 tratam `FORCE` explicitamente como o padrão da casa ("sem FORCE, o OWNER da tabela ignora a política"); estas 12 são dívida, não decisão.

IMPACTO

Sob uma configuração de banco plausível e não bloqueada por nada, a segunda camada de defesa contra vazamento entre organizações some das quatro tabelas mais sensíveis do produto (missões, ledger de XP, empresas-cliente e fechamentos), restando só o filtro por `org_id` na aplicação.

CONDIÇÕES DE EXPLORAÇÃO

Condicional: exige que `DATABASE_URL` aponte para o role dono/superuser das tabelas. Não é o padrão dos arquivos de exemplo, mas é o modo de implantação que `start-production.mjs` prevê e apenas adverte por log.

CORRIGIDO REMEDIAÇÃO

O que foi feito: Migration 0062 aplica `FORCE ROW LEVEL SECURITY` nas 12 tabelas do núcleo, alinhando ao padrão das demais 30.

Verificação: No banco: 42/42 tabelas com `org_id` agora `enabled=true forced=true`, zero pendências. Ataque como `guilda_app` na org do atacante contra linhas de outro tenant: 0 leituras e 0 escritas.

MÉDIA

CORRIGIDO

F15 Notificações do Telegram sobreviviam à saída do integrante da organização

src/lib/clans/bootstrap.ts:236 (cleanup do offboarding)

src/lib/telegram/notifications.ts:115 (broadcast)

src/lib/telegram/worker.ts:310 (resumo/lembrete agendado)

```
// offboarding ANTES: só apagava o vínculo de clã, nada tocava a conexão do bot
await tx.delete(schema.clanMemberships).where(/* org + user */);
// (telegram_connections continuava ativa)

// e as DUAS rotas de envio enumeram conexões SEM provar vínculo:
const connections = await tx.query.telegramConnections.findMany({
  where: and(eq(...orgId), isNull(...revokedAt)), // <- não checa 'member'
});
```

POR QUE É EXPLORÁVEL

`cleanupRemovedOrganizationMemberClans` removía apenas as linhas de `clan_memberships` ao tirar alguém da organização. A conexão do bot (`telegram_connections`) continuava ativa, e as duas rotas de entrega — o broadcast (`enqueueTelegramOrgBroadcast`) e o agendador do worker — enumeram conexões filtrando só por `org_id` e `revoked_at`, sem provar que a pessoa ainda é `member`. Resultado: um ex-integrante com Telegram vinculado seguia recebendo todo aviso do Mural, toda mudança de fechamento **com o nome da empresa-cliente** e o resumo diário, por tempo indeterminado. O bot recusava comandos dele (o handler checa `member`), mas a entrega passiva continuava.

IMPACTO

Vazamento contínuo de informação operacional e de nomes de empresas-cliente para quem já saiu da organização, até que a conexão fosse revogada manualmente — o que nada fazia.

CONDIÇÕES DE EXPLORAÇÃO

Ter o Telegram vinculado antes de ser removido da organização. A entrega segue automática, sem ação do ex-integrante.

CORRIGIDO

REMEDIÇÃO

O que foi feito: O offboarding revoga a conexão do Telegram e invalida tokens pendentes; as duas rotas de envio ganharam INNER JOIN member — quem saiu para de receber.

Verificação: No banco: após remover o membro, a consulta antiga ainda alcançaria a conexão (1), a nova (com join) alcança 0.

INFORMATIVA

CORRIGIDO

F5 O verificador de RLS não cobre as tabelas do núcleo

scripts/check-rls.mjs:13-38

scripts/check-rls.mjs:71-75

```
// scripts/check-rls.mjs:13-38 - a lista auditada por `npm run check:rls`
const NEW_TABLES = [
  "clan_informative_routes", "task_assignee_suggestions", "guild_notices",
  "guild_notice_reads", "informatives", "fiscal_portfolios", /* ... */
  "company_flows", "company_flow_secrets", "company_flow_events",
];
// tasks, task_events, xp_ledger, clients e accounting_closings não estão aqui.

// :71-75 - a checagem exige enabled E forced, mas só para NEW_TABLES
check(t, Boolean(row?.relrowsecurity && row?.relforcerowsecurity), ...);
```

POR QUE É EXPLORÁVEL

O script exige corretamente `relrowsecurity && relforcerowsecurity`, mas só para as 24 tabelas “novas”. As tabelas do núcleo nunca entraram na lista, e é por isso que a dívida de `FORCE` do F4 atravessou 60 migrations sem ser notada — a suíte que existia justamente para pegar esse tipo de regressão estava olhando para o outro lado.

IMPACTO

Falsa sensação de cobertura: `npm run check:rls` passa com sucesso mesmo com `tasks`, `xp_ledger`, `clients` e `accounting_closings` sem `FORCE`.

CONDIÇÕES DE EXPLORAÇÃO

Não explorável por si. É o motivo pelo qual F4 permaneceu invisível.

CORRIGIDO**REMEDIÇÃO**

O que foi feito: check-rls.mjs deriva a lista de pg_class (toda tabela com coluna org_id) em vez de uma constante fixa, e exige relrowsecurity AND relforcerowsecurity. Tabela nova entra na checagem sozinha.

Verificação: npm run check:rls roda contra o banco e reporta as 42 tabelas; TODAS AS CHECAGENS PASSARAM, com provas de isolamento incluídas para tasks, xp_ledger e clients.

Categoria 2 Permissão definida no navegador

Padrão sólido e repetido — os fatos de autorização (papel na organização, liderança do clã) são lidos do banco dentro da própria action e passados a funções puras testadas. Duas exceções quebram esse padrão: o arquivo de Fechamentos, que não tem gate nenhum, e a flag de troca obrigatória de senha, que é limpa pelo navegador.

ALTA**CORRIGIDO****F1 Sete Server Actions de Fechamentos sem verificação de clã ou papel**`src/app/(app)/clans/[id]/closing-actions.ts:88 (createClosing)``src/app/(app)/clans/[id]/closing-actions.ts:146 (updateClosing)``src/app/(app)/clans/[id]/closing-actions.ts:214 (setClosingStatus)``src/app/(app)/clans/[id]/closing-actions.ts:281 (deleteClosing)``src/app/(app)/clans/[id]/closing-actions.ts:440 (setDefisCompleted)``src/app/(app)/clans/[id]/closing-actions.ts:506 (updateYearNotes)`

```
// closing-actions.ts:281 - deleteClosing, íntegra do controle de acesso
export async function deleteClosing(
  input: z.input<typeof deleteClosingSchema>,
): Promise<ActionResult> {
  const ctx = await requireMemberContext(); // <- só prova que é membro da org
  if (!ctx.ok) return ctx;

  const parsed = deleteClosingSchema.safeParse(input);
  if (!parsed.success) return err("Fechamento inválido.");

  const deleted = await withOrgTx(ctx.orgId, (tx) =>
    tx.delete(schema.accountingClosings).where(
      and(
        eq(schema.accountingClosings.id, parsed.data.closingId),
        eq(schema.accountingClosings.orgId, ctx.orgId),
      ),
    ).returning({ id: schema.accountingClosings.id }),
  );
};
```

POR QUE É EXPLORÁVEL

`requireMemberContext()` prova apenas que quem chamou é membro da organização — não checa clã nem papel. A restrição “Fechamentos pertence à Contabilidade” mora exclusivamente na navegação: `src/lib/klan-tabs.ts:37` só oferece a aba `closings` ao clã de slug `contabilidade`, e `src/app/(app)/clans/[id]/page.tsx:133` devolve 404 para quem não é do clã. Nenhuma dessas duas barreiras existe no caminho da Server Action. Como `closing-board.tsx:1` é um Client Component que importa essas actions (:44-47), os identificadores `Next-Action` ficam em um chunk estático público em `/_next/static/chunks/`, acessível a qualquer usuário autenticado. Um `member` do clã Fiscal (ou de qualquer outro) pode então apagar, criar e reescrever fechamentos contábeis de qualquer empresa da organização. O contraste interno é a prova de que isso é lapso e não decisão: o arquivo irmão da mesma aba, `commitment-actions.ts`, protege as suas 8 actions com `requireDistributionManager` (:45-67), que valida o slug do clã e chama `canManageClanCommitments`.

IMPACTO

Perda e adulteração silenciosa de registros contábeis (fechamentos, situação, saldo de caixa, resultado, empréstimo de sócio, observações e DEFIS) de qualquer uma das empresas-cliente, por qualquer pessoa com conta na organização.

CONDIÇÕES DE EXPLORAÇÃO

Requer apenas uma conta ativa de `member` na organização. Nenhuma feature flag, nenhuma configuração insegura.

CORRIGIDO

REMEDIÇÃO

O que foi feito: Gate `requireClosingManager/requireClosingActor` nas 7 actions, espelhando `requireDistributionManager`: exige clã Contabilidade e papel, com `clanId` nos schemas Zod. Régua pura `canManageClanClosings/canDeleteClanClosing` em `guild-permissions.ts`, com testes.

Verificação: Pentest HTTP: `member` sem vínculo recebe “Apenas quem integra a Contabilidade...” e nada é gravado; `owner` segue conseguindo. Aba renderiza normalmente. tsc, lint, 510 testes e build passam.

ALTA

MITIGADO

F2 setYearClosed credita XP para quem chamou, sem gate — farm de XP em massa`src/app/(app)/clans/[id]/closing-actions.ts:316-397`

```
// closing-actions.ts:384-396
let xpAwarded = false;
if (data.closed) {
  const credited = await tx
    .insert(schema.xpLedger)
    .values({
      orgId: ctx.orgId,
      userId: ctx.userId,          // <- o próprio chamador recebe o XP
      closingYearId: annual.id,
      amount: CLOSING_YEAR_XP,
      reason: "closing_year_closed",
    })
    .onConflictDoNothing()
    .returning({ id: schema.xpLedger.id });
  xpAwarded = credited.length > 0;
}
```

POR QUE É EXPLORÁVEL

Mesma ausência de gate do F1, mas com um efeito colateral que sai do domínio contábil e cai na gamificação. A action aceita qualquer `clientId` da organização e qualquer `year` entre 2000 e 2100 (:309) e credita `CLOSING_YEAR_XP` ao usuário da sessão. A idempotência existe (índice único parcial `xp_ledger_closing_year_closed_uidx` em `src/db/migrations/0015_thankful_runaways.sql:3`), mas ela é por par (empresa, ano) — não por pessoa. Com ~250 empresas-cliente e 101 anos aceitos, um único `member` pode gerar mais de 25 mil créditos distintos em laço, todos legítimos aos olhos do banco. O `xp_ledger` é imutável por privilégio (`REVOKE UPDATE, DELETE` em `0004_rls-xp-ledger.sql:10`), o que aqui joga contra: os lançamentos falsos não podem ser apagados, só estornados um a um.

IMPACTO

Destruição da integridade do ranking e do sistema de níveis — o mecanismo central do produto. Efeito colateral: fecha anos contábeis de empresas arbitrárias, o que dispara notificação para a organização inteira (:400-419) e bloqueia o registro da DEFIS (:472).

CONDIÇÕES DE EXPLORAÇÃO

Conta ativa de `member`. O laço é trivial: para cada empresa da organização, chamar a action com `closed: true` variando o ano.

MITIGADO

REMEDIÇÃO

O que foi feito: O gate do F1 remove o acesso de fora da Contabilidade e de outros tenants. A trava adicional de XP (limitar a faixa de anos ou creditar só pela missão) NÃO foi adicionada por decisão consciente: o farm de XP DENTRO da Contabilidade continua possível.

Verificação: Pentest: `setYearClosed` cross-clã/cross-tenant bloqueado, 0 XP creditado. O resíduo intra-clã é aceito; auditoria por `closed_by_task_id` separa o manual do legítimo. Reavaliar se o ranking degradar.

MÉDIA

CORRIGIDO

F3 A troca obrigatória de senha temporária é desativada pelo próprio navegador`src/lib/auth.ts:25-33``src/app/(auth)/change-password/change-password-form.tsx:54``src/lib/session.ts:31-41`

```
// src/lib/auth.ts:25-33 - campo adicional marcado como gravável pelo cliente
user: {
  additionalFields: {
    mustChangePassword: {
      type: "boolean",
      defaultValue: false,
      input: true,          // <- aceito no corpo de /update-user
    },
  },
},

// src/app/(auth)/change-password/change-password-form.tsx:54
// Marca a troca como feita - deixa de ser redirecionado para cá.
await authClient.updateUser({ mustChangePassword: false });
```

POR QUE É EXPLORÁVEL

`createMemberWithTempPassword` (`src/app/(app)/members/actions.ts:61`) cria a conta com `mustChangePassword: true`, e `requireOrgSession` (`src/lib/session.ts:33-35`) redireciona para `/change-password` enquanto a flag estiver ligada. Só que a flag é declarada com `input: true`, e o better-auth aceita campos adicionais assim marcados no corpo do endpoint `/update-user` — confirmado em `node_modules/better-auth/dist/api/routes/update-user.mjs:54`, `parseUserInput(ctx.context.options, rest, "update")`. Quem baixa a flag hoje é o navegador, numa segunda chamada logo depois de trocar a senha; nada no servidor amarra o desligamento da flag ao fato de a senha ter sido efetivamente trocada. Qualquer usuário autenticado pode emitir `POST /api/auth/update-user {"mustChangePassword": false}` e continuar operando indefinidamente com a senha temporária que o admin distribuiu por canal externo. Como agravante, as Server Actions usam `requireMemberContext` (`src/lib/action-context.ts:23-49`), que não olha a flag — logo, mesmo sem desligá-la, a pessoa já consegue executar qualquer mutação; a barreira do `/change-password` só existe na navegação de páginas.

IMPACTO

Senhas temporárias compartilhadas fora do sistema (chat, e-mail, papel) permanecem válidas por tempo indeterminado, aumentando a janela para tomada de conta de qualquer membro — inclusive de um `admin` recém-criado.

CONDIÇÕES DE EXPLORAÇÃO

Uma chamada autenticada ao endpoint do better-auth. Não depende de nenhuma configuração especial.

CORRIGIDO REMEDIAÇÃO

O que foi feito: `mustChangePassword` passou a `input:false` (rejeitado em `/update-user`); a flag só é desligada pelo servidor, como efeito de `changeOwnPassword`. `requireMemberContext` também exige a troca, cobrindo as Server Actions.

Verificação: Pentest: `POST /update-user {mustChangePassword:false}` → 400 "No fields to update", flag permanece; Server Action com flag ligada → "Defina uma senha própria antes de continuar."; troca legítima desliga a flag e a senha antiga passa a dar 401.

Categoria 3 IDOR e roteamento por parâmetro

Nenhum IDOR de objeto-por-id: os 86 handlers resolvem o objeto com `and(eq(tabela.id, ...), eq(tabela.orgId, ctx.orgId))` dentro de `withOrgTx`, e o pentest cross-tenant confirmou 0 leituras e 0 escritas de outro tenant. Aqui entram dois achados de parâmetro de rota: o F6 (matcher do proxy, cosmético) e o F14, um open redirect no `?next=` do sign-in contornável por barra invertida — corrigido nesta revisão.

MÉDIA

CORRIGIDO

F14 Filtro de open redirect do sign-in contornável por barra invertida e caracteres de controle

`src/app/(auth)/sign-in/page.tsx:14 (versão anterior)`

```
// versão anterior - checagem TEXTUAL
const safeNext =
  next?.startsWith("/") && !next.startsWith("//") ? next : undefined;
// ...depois: router.push(next ?? "/dashboard")

// o parser de URL resolve estes para OUTRA origem, mas passavam no filtro:
new URL("/\\evil.com", base).href // -> https://evil.com/ (barra invertida = barra)
new URL("/\t/evil.com", base).href // -> https://evil.com/ (tab é removido antes do parse)
```

POR QUE É EXPLORÁVEL

O destino do parâmetro `?next=` era validado por texto (começa com `/` e não com `//`) e depois passado direto para `router.push`. Dois payloads escapam dessa regra porque o parser de URL os resolve para outra origem: `/\\evil.com` (a barra invertida equivale a barra em esquemas especiais) e `/<tab>/evil.com` (tab, CR e LF são removidos antes do parse). O atacante monta `.../sign-in?next=\\evil.com`, a vítima faz login legítimo e é levada para fora do domínio — primitivo clássico de phishing, com a credibilidade de ter partido do domínio real da Guilda.

IMPACTO

Redirecionamento pós-login para domínio controlado pelo atacante, a partir de um link que começa no domínio confiável. Facilita phishing e roubo de sessão em páginas-isca.

CONDIÇÕES DE EXPLORAÇÃO

Enviar à vítima um link de login com `?next=` malicioso. Sem pré-condição de configuração.

CORRIGIDO

REMEDIÇÃO

O que foi feito: O destino do `?next=` passou a ser resolvido contra uma origem sentinela e comparado por origem (padrão de `returnToTasks`), em vez de checagem textual.

Verificação: 13 testes em `safe-redirect.test.ts` cobrindo os dois bypasses. Pentest: login com `?next=\\example.com` termina em `/dashboard`.

INFORMATIVA

CORRIGIDO

F6 O matcher do proxy não acompanha as rotas protegidas do app

`src/proxy.ts:30-43`

```
// src/proxy.ts:30-43
export const config = {
  matcher: [
    "/dashboard/:path*", "/tasks/:path*", "/clients/:path*",
    "/campaigns/:path*", "/leaderboard/:path*", "/members/:path*",
    "/profile/:path*", "/onboarding", "/change-password",
    "/sign-in", "/sign-up",
  ],
};
// Ausentes: /clans, /settings, /informativos, /mural, /closings
```

POR QUE É EXPLORÁVEL

Cinco rotas do grupo `(app)` não passam pelo proxy — inclusive `/settings`, que é a tela de administração dos clãs. **Isto não abre acesso:** `src/app/(app)/layout.tsx:13` chama `requireOrgSession()`, que valida a sessão de verdade no servidor e redireciona para `/sign-in`, e o próprio `proxy.ts:5-8` documenta que a checagem dele é apenas otimista. O efeito real é de custo e de sinal: um visitante não autenticado renderiza o layout antes de ser mandado embora, e a lista congelada é um convite a alguém no futuro presumir que “o matcher é a proteção” e criar uma rota que dependa disso.

IMPACTO

Nenhum acesso indevido. Redirecionamento mais lento nas rotas ausentes e risco de manutenção.

CONDIÇÕES DE EXPLORAÇÃO

Não explorável — a proteção real está no layout do servidor.

CORRIGIDO REMEDIAÇÃO

O que foi feito: O matcher do proxy passou a cobrir as 5 rotas que faltavam (clans, closings, informativos, mural, settings), e ganhou um comentário forte de que é OTIMIZAÇÃO, não a fronteira de segurança — endereçando o risco de manutenção. Evitou-se o catch-all negativo de propósito: ele redirecionaria os próprios ícones/manifest para `/sign-in`.

Verificação: tsc, lint e 527 testes passam; build compila o proxy. A proteção real continua no layout (`requireOrgSession`), já confirmada no pentest (sem sessão → 307).

Categoria 4 Chaves expostas (hardcode)

Nenhuma chave real de produção no código, nos configs ou no histórico do Git — só credenciais de demonstração/desenvolvimento. Na remediação ganharam trava: `env-guard` recusa segredos fracos/placeholder no boot de produção (F9/F10), o seed aborta fora do ambiente local (F7), a porta do Postgres de dev ficou em loopback (F8) e a planilha com dados de cliente foi apagada e barrada no `.gitignore` (F11).

MÉDIA

CORRIGIDO

F7 Seed cria uma conta owner com senha fixa, sem trava de ambiente, e vai para a imagem de produção`scripts/seed.ts:7``scripts/seed.ts:21``scripts/seed.ts:78``scripts/screenshots.mjs:17``README.md:113``Dockerfile:42`

```
// scripts/seed.ts:7,21,24-29,78
* Login demo: helena@demo.gilda.dev / demo123456 (e demais e-mails)
const PASSWORD = "demo123456";
const PEOPLE = [
  { key: "helena", name: "Helena Prado", email: "helena@demo.gilda.dev", role: "owner" },
  ...
];
await auth.api.signUpEmail({ body: { name, email, password: PASSWORD } });

# Dockerfile:42 - scripts/ entra na imagem final de runtime
COPY --from=build --chown=nextjs:nodejs /app/scripts ./scripts
```

POR QUE É EXPLORÁVEL

Senha fixa de 10 caracteres criando conta com papel `owner`, documentada no `README.md:113` e repetida em `scripts/screenshots.mjs:17`. Não existe guarda de `NODE_ENV`: a única condição de parada é a organização `gilda-demo` já existir (:87-94), o que não impede a primeira execução contra um banco de produção. O `.dockerignore` exclui `scripts/e2e-*.mjs` (:9) mas não `scripts/seed.ts`, e o `Dockerfile:42` copia o diretório inteiro para a imagem final — que também tem `tsx` disponível via `node_modules` e o `package.json` com o alvo `npm run seed` (:16). Um `docker exec` no contêiner de produção, ou um erro de operação na VPS, cria owner com credencial pública em uma organização real.

IMPACTO

Criação de conta `owner` com credencial de conhecimento público em ambiente produtivo — controle total de um tenant, incluindo leitura das senhas Gov.br descriptografadas via `revealCompanyFlowGovPassword`.

CONDIÇÕES DE EXPLORAÇÃO

Exige que alguém execute `npm run seed` contra o banco de produção. Nada no código impede; o caminho está pronto dentro da imagem publicada.

CORRIGIDO

REMEDIÇÃO

O que foi feito: Seed aborta se `NODE_ENV=production` ou se a `DATABASE_URL` não for local (host parseado), com `escape ALLOW_SEED=1`; senha vem de `SEED_PASSWORD`. `.dockerignore` exclui `seed.ts`, `screenshots.mjs`, `check-rls.mjs` e `generate-app-icons.mjs`.

Verificação: Os três caminhos da trava exercitados: produção → bloqueado; URL remota → bloqueado; `ALLOW_SEED=1` → segue. Seed contra o banco local funciona.

BAIXA

CORRIGIDO

F8

Senha do role da aplicação fixa no init do Postgres de desenvolvimento

docker/dev/init/01-roles.sql:5

.env.example:3

.env.example:5

docker-compose.dev.yml:9

docker-compose.dev.yml:11-12

```
-- docker/dev/init/01-roles.sql:5
CREATE ROLE guilda_app LOGIN PASSWORD 'guilda_app_dev' NOSUPERUSER NOCREATEDB NOCREATEROLE;

# .env.example:3,5
DATABASE_URL=postgresql://guilda_app:guilda_app_dev@localhost:5432/guilda
MIGRATION_DATABASE_URL=postgresql://postgres:postgres@localhost:5432/guilda

# docker-compose.dev.yml:9,11-12
  POSTGRES_PASSWORD: postgres
  ports:
    - "5432:5432"
```

POR QUE É EXPLORÁVEL

Credenciais literais de desenvolvimento (`guilda_app_dev` , `postgres:postgres`) versionadas, com a porta 5432 publicada no host sem restrição de interface. O equivalente de produção está correto — `docker/prod/init/01-roles.sh:7` interpola `${APP_DB_PASSWORD}` e `docker-compose.yml:15` exige a variável com `${APP_DB_PASSWORD:?...}` —, então o risco é só de ambiente local: em rede compartilhada, ou se alguém copiar `docker-compose.dev.yml` para uma VPS, essas credenciais são conhecidas.

IMPACTO

Acesso ao banco de desenvolvimento por qualquer um na mesma rede. Sem impacto direto em produção.

CONDIÇÕES DE EXPLORAÇÃO

Requer alcance de rede à porta 5432 da máquina de desenvolvimento, ou reaproveitamento do Compose de dev fora do local.

CORRIGIDO

REMEDIACÃO

O que foi feito: A porta do Postgres de dev passou a ligar em 127.0.0.1:5432 — não fica mais exposta à rede. As credenciais triviais de dev seguem, mas atrás do loopback ninguém na rede as alcança.

Verificação: `docker-compose.dev.yml` aponta 127.0.0.1:5432:5432; a app conecta pelo localhost do host normalmente (fluxo de dev inalterado).

BAIXA

CORRIGIDO

F9

Placeholder literal de BETTER_AUTH_SECRET no Dockerfile, sem validação própria de inicialização

Dockerfile:16-18

src/lib/auth.ts:18-24

```
# Dockerfile:14-18 - estágio 'build'
# Placeholders APENAS para o build (todas as páginas são dinâmicas -
# nenhuma conexão acontece). Os valores reais entram em runtime.
ENV DATABASE_URL=postgresql://build:build@localhost:5432/build
ENV BETTER_AUTH_SECRET=placeholder-somente-para-o-build-32ch
ENV BETTER_AUTH_URL=http://localhost:4000
```

POR QUE É EXPLORÁVEL

Verificado: o estágio `runner` parte de `FROM base` (`Dockerfile:32`) e **não** herda os `ENV` do estágio `build`, então este valor não vira o segredo de runtime. Restam dois pontos. Primeiro, o literal tem 36 caracteres e não é o `DEFAULT_SECRET` do `better-auth` — passaria pelas duas travas da biblioteca (`node_modules/better-auth/dist/context/create-context.mjs:41-44`, que só lança para o default exato e para segredo ausente) caso um dia fosse promovido a runtime por engano. Segundo, o projeto não tem validação própria de inicialização que rejeite segredos fracos ou conhecidos; ele depende inteiramente do que o `better-auth` decidir checar. Considerando que o comentário do próprio `Dockerfile` (`:49-50`) registra que o painel de hospedagem constrói só o `Dockerfile`, sem o `Compose`, o `${VAR:?...}` do `Compose` — que é a trava real do projeto — não roda nesse caminho de implantação.

IMPACTO

Nenhum vazamento hoje. É a ausência de uma trava: no caminho de deploy por painel, nada além da biblioteca impede subir com um segredo fraco.

CONDIÇÕES DE EXPLORAÇÃO

Não explorável na configuração atual. Torna-se relevante se o placeholder for promovido a variável de runtime ou se um segredo fraco for configurado no painel.

CORRIGIDO REMEDIAÇÃO

O que foi feito: Novo `src/lib/env-guard.ts` validado no boot por `src/instrumentation.ts`: recusa `BETTER_AUTH_SECRET` curto, placeholders (`Dockerfile`, `better-auth`, `.env.example`) e valores de baixa entropia. Em produção derruba o boot; fora dela só avisa; a fase de `build` é ignorada (`NEXT_PHASE`).

Verificação: Testes cobrem placeholders, entropia e o ramo lançar-vs-avisar. Build de produção não quebra (skip na fase de build); boot de produção com o placeholder do `Dockerfile` lança e não sobe.

INFORMATIVA

CORRIGIDO

F10 `FLOW_SECRETS_KEY` aceita qualquer valor que decodifique para 32 bytes

`src/lib/company-flows/secrets.ts:13-20`

```
// src/lib/company-flows/secrets.ts:13-20
function encryptionKeyFromEnvironment(): Buffer | null {
  const value = process.env.FLOW_SECRETS_KEY?.trim();
  if (!value) return null;
  const key = /^[A-Za-f0-9]{64}$/.test(value)
    ? Buffer.from(value, "hex")
    : Buffer.from(value, "base64");
  return key.length === 32 ? key : null; // <- só o tamanho é conferido
}
```

POR QUE É EXPLORÁVEL

A criptografia em si está correta: AES-256-GCM com IV aleatório por registro e `authTag` verificado (`:22-48`). O que falta é qualquer aferição de qualidade da chave — um valor previsível de 32 bytes é aceito igual a um gerado por `openssl rand`. Como essa chave protege senhas Gov.br de empresas-cliente (`companyFlowSecrets`), o dado protegido tem valor alto o bastante para justificar a checagem. O comportamento de falha, por outro lado, está certo: sem chave, a aplicação recusa gravar a senha em vez de guardá-la em claro (`company-flow-actions.ts:295-298`).

IMPACTO

Chave fraca configurada por engano protege senhas de acesso a portal de governo sem que nada avise.

CONDIÇÕES DE EXPLORAÇÃO

Depende de configuração fraca do operador.

CORRIGIDO REMEDIAÇÃO

O que foi feito: O mesmo `env-guard` valida `FLOW_SECRETS_KEY`: se definida, precisa decodificar para 32 bytes E ter entropia (recusa chave degenerada como tudo-zero). Vazia continua aceita (feature opcional).

Verificação: Testes: chave forte base64/hex passa, 'tudo zero' é recusada por entropia, vazia é aceita. Em produção uma chave fraca derruba o boot.

INFORMATIVA

CORRIGIDO

F11 Planilha com dados do escritório na raiz do repositório, sem regra no .gitignore`.gitignore``Planilha contole de Notas do escritório.xlsx`

```
$ git status --short
?? .impeccable/
?? "Planilha contole de Notas do escritório.xlsx"

# .gitignore não tem nenhuma regra para planilhas:
# (nenhuma linha *.xlsx / *.xls / *.csv)
```

POR QUE É EXPLORÁVEL

O arquivo está apenas não rastreado — não foi commitado, e a varredura do histórico do Git não encontrou nenhum `.env`, chave privada ou token em nenhum commit. Mas nada impede que um `git add .` o inclua, e o nome sugere dados fiscais reais de empresas-cliente. É o tipo de arquivo que costuma entrar num repositório por acidente e depois é impossível remover do histórico sem reescrevê-lo.

IMPACTO

Risco de exposição futura de dados de clientes caso o arquivo seja commitado — irreversível no histórico se o repositório for público ou compartilhado.

CONDIÇÕES DE EXPLORAÇÃO

Não há exposição hoje. É prevenção.

CORRIGIDO

REMEDIÇÃO

O que foi feito: A planilha com dados reais de clientes (CNPJ, honorários) foi APAGADA do repositório, e `*.xlsx/*.xls/*.csv` entraram no `.gitignore` para impedir um `git add .` acidental no futuro.

Verificação: `git status` limpo (arquivo removido); `git check-ignore` confirma a regra; histórico do Git seguia e segue sem nenhuma planilha.

Categoria 5 Inputs sem tratamento (XSS)

Nenhum sink de XSS no projeto — nem no cliente nem no servidor. Os dois pontos de endurecimento foram fechados: a CSP passou a usar nonce + 'strict-dynamic' em produção, sem 'unsafe-inline' (F12, verificado no navegador), e a armadilha do escape sem consumidor do Telegram foi removida (F13).

BAIXA

CORRIGIDO

F12

CSP depende de 'unsafe-inline' em script-src

next.config.ts:8-20

```
// next.config.ts:8-12
const contentSecurityPolicy = [
  "default-src 'self'",
  "script-src 'self' 'unsafe-inline'", // <- anula boa parte da CSP
  "style-src 'self' 'unsafe-inline'",
  ...
```

POR QUE É EXPLORÁVEL

O comentário logo acima (:3-7) já reconhece a limitação e aponta o caminho (nonce via proxy). Com 'unsafe-inline' em script-src, a CSP deixa de ser uma segunda linha de defesa contra XSS — se um sink aparecer amanhã, o cabeçalho não segura. Como a auditoria não encontrou nenhum sink hoje (nenhum dangerouslySetInnerHTML, innerHTML, eval, new Function, srcDoc ou href com esquema controlado pelo usuário em todo src/), isto é endurecimento e não correção de vulnerabilidade. Vale registrar o que está certo: object-src 'none', base-uri 'self', form-action 'self', frame-ancestors 'none' e a ausência de 'unsafe-eval'.

IMPACTO

Perda da mitigação em profundidade contra XSS. Sem impacto isolado, já que não há sink no código atual.

CONDIÇÕES DE EXPLORAÇÃO

Não explorável sozinho — depende da introdução futura de um sink.

CORRIGIDO

REMEDIACÃO

O que foi feito: A CSP saiu do next.config (estática) e virou por-requisição no proxy: em produção o script-src usa nonce + 'strict-dynamic', SEM 'unsafe-inline'. Em dev a política é afrouxada para o HMR/overlay não quebrar. Os demais cabeçalhos (HSTS, nosniff, etc.) seguem estáticos para todas as rotas.

Verificação: Em next start (produção): o header de /sign-in traz script-src 'self' 'nonce-...' 'strict-dynamic' (nonce diferente a cada requisição, sem unsafe-inline). No navegador a página renderiza, hidrata e faz login (fetch do cliente) sob a CSP estrita, sem violações no console.

INFORMATIVA

CORRIGIDO

F13

Função de escape do Telegram sem consumidor — armadilha latente

src/lib/telegram/format.ts:3-9

src/lib/telegram/endpoint.ts:58

src/lib/telegram/endpoint.ts:115

```
// src/lib/telegram/format.ts:3-9 - existe, mas nenhum código de produção chama
/** Escapa texto não confiável para 'parse_mode: HTML' do Telegram. */
export function escapeTelegramHtml(value: string): string {
  return value.replaceAll("&", "&amp;").replaceAll("<", "&lt;").replaceAll(">", "&gt;");
}

// src/lib/telegram/endpoint.ts:115 - parse_mode só é enviado se alguém pedir
...(options?.parseMode ? { parse_mode: options.parseMode } : {}),
```

POR QUE É EXPLORÁVEL

Nenhum chamador em `src/` define `parseMode`, então todas as mensagens saem como texto puro e não há injeção possível hoje — títulos de missão, notas de rejeição e nomes de empresa entram nas mensagens sem escape (`tasks/actions.ts:443,460,472`; `telegram/actions.ts:153,170,182`) e isso está correto para texto puro. O risco é de manutenção: a função de escape existe, o tipo `parseMode?: "HTML"` existe, e a documentação implícita sugere que basta ligar o `parse_mode`. Quem fizer isso sem passar por `escapeTelegramHtml` cria injeção de marcação em mensagens que a organização inteira recebe.

IMPACTO

Nenhum hoje. Injeção de HTML nas notificações do Telegram se `parse_mode` for ativado sem aplicar o escape.

CONDIÇÕES DE EXPLORAÇÃO

Não explorável na configuração atual.

CORRIGIDO REMEDIAÇÃO

O que foi feito: A armadilha foi REMOVIDA: apagados a função `escapeTelegramHtml`, o tipo `parseMode` (`endpoint.ts` e `types.ts`) e o ramo condicional de `parse_mode`. Um comentário no lugar explica que as mensagens são texto puro e o que fazer se um dia alguém ligar HTML.

Verificação: Nada mais referencia `parseMode/escapeTelegramHtml`; tsc, lint e 527 testes passam. Mensagens do bot seguem como texto puro (comportamento inalterado).

Validação ofensiva — teste de intrusão

Após as correções, rodou-se um teste de intrusão ativo assumindo o papel de um invasor autenticado com o MENOR privilégio (um member sem vínculo de clã) contra um tenant-vítima real, plantado com dados sensíveis rastreáveis — incluindo uma senha Gov.br cifrada com o próprio código do projeto. Nenhuma vulnerabilidade nova foi encontrada: as cinco classes de ataque foram contidas.

IDOR / isolamento entre tenants

BLOQUEADO

Método: Conexão como guilda_app (o exato role da aplicação) com app.org_id da org do atacante, atacando as linhas do outro tenant por ID — o mesmo que uma Server Action faz após autenticar. Prova determinística e mais forte que o teste HTTP: nenhuma action pode ir além do que este role alcança.

Evidência: Cliente, fechamento, fluxo e a senha Gov.br do outro tenant: 0 leituras. UPDATE de nome, DELETE de fechamento e DELETE de cliente: 0 linhas afetadas.

Escalação de privilégio

BLOQUEADO

Método: Como member, via HTTP (IDs de action extraídos do bundle, como um atacante faria): criar conta admin, virar líder de clã, auto-adicionar-se a um clã, criar clã.

Evidência: Criar admin → “Apenas admin ou owner pode adicionar membros.” (0 contas criadas). Líder/adicionar/criar clã → “Apenas admin ou owner pode gerenciar a composição dos clãs.”

Injeção de XP / mass-assignment

DERROTADO

Método: createTask injetando xpValue:999999, status:“completed”, orgId de outro tenant e creatorId falso no corpo.

Evidência: Gravado: xp_value=120 (computado pelo servidor), status=pending, org= a da sessão, creator= o da sessão, 0 lançamentos no ledger. Zod descarta chaves desconhecidas; o servidor fixa o que é sensível.

Ciclo de vida da tarefa (auto-aprovação / roubo de XP)

BLOQUEADO

Método: Responsável tenta auto-aprovar a própria tarefa de terceiro; outro member tenta concluir a tarefa alheia; member tenta reverter conclusão (só admin).

Evidência: Auto-aprovação → “Apenas quem criou a missão ou um admin pode aprovar/rejeitar.”; concluir tarefa alheia → “Apenas a pessoa responsável pode concluir a missão.”; reversão por member → negada. 0 XP indevido.

Superfície anônima

LIMPO

Método: Webhook do Telegram sem/errado segredo; acesso a páginas de dados sem sessão.

Evidência: Webhook → 503 (inerte sem token). /clients, /settings, /clans/[id], /members sem cookie → 307 para /sign-in. Sem vazamento.

O ambiente de teste (dev server + Docker sobre OneDrive/Windows) caiu várias vezes sob compile pesado — instabilidade de infraestrutura, não da aplicação. Contornado rodando o servidor destacado e provando o isolamento pelo role real guilda_app. As classes de lógica de app foram provadas por HTTP de verdade.

Correção sobre a F4 — o que FORCE realmente protege

Medido no banco de dev, com app.org_id apontando para uma org inexistente: como postgres (dono das tabelas E superuser) veem-se 42 missões; como guilda_app (não-superuser), 0. FORCE ROW LEVEL SECURITY só alcança o dono NÃO-superuser. Como aqui o dono é o postgres superuser, nem ENABLE nem FORCE o contém — num deploy com esse role, o isolamento sumiria nas 42 tabelas, não só nas 12. A leitura correta: a proteção real é a aplicação conectar como guilda_app; o RLS/FORCE é a segunda camada e só vale contra dono não-superuser (o cenário de um Postgres gerenciado, que a migration 0062 passa a cobrir).

Recomendações priorizadas

Registradas na 1ª rodada como plano de ação. Nesta revisão **todas — P1, P2 e P3 — foram executadas e verificadas** (ver o estado de cada achado). Ficam aqui como registro do que foi feito e em que ordem.

P1 Imediato — antes do próximo deploy

1. Criar `requireClosingManager(tx, ctx, clanId)` em `closing-actions.ts`, espelhando `requireDistributionManager` de `commitment-actions.ts:45-67`: validar `clan.slug === CONTABILIDADE_CLAN_SLUG` e chamar uma régua de domínio (`canManageClanClosings`) antes de qualquer escrita. Aplicar às 7 actions e acrescentar `clanId` aos schemas Zod. cobre F1, F2
2. Auditar o `xp_ledger` em busca de lançamentos `closing_year_closed` sem fechamento correspondente legítimo e estornar com `reason = 'reversal'` (nunca DELETE — o role não tem esse privilégio). cobre F2

P2 Próximo ciclo

1. Migration aplicando `FORCE ROW LEVEL SECURITY` às 12 tabelas restantes (`tasks`, `task_events`, `xp_ledger`, `clients`, `accounting_closings`, `accounting_closing_years`, `mission_templates`, `mission_template_items`, `telegram_connections`, `telegram_link_tokens`, `telegram_preferences`, `telegram_outbox`), seguindo o padrão de `0025_force-rls-informatives.sql`. cobre F4
2. Trocar a lista fixa `NEW_TABLES` de `check-rls.mjs` por uma consulta a `pg_class` que exija `relrowsecurity AND relforcerowsecurity` em toda tabela com coluna `org_id` — assim a checagem passa a cobrir tabelas futuras sozinha. cobre F5
3. Remover `input: true` de `mustChangePassword` em `src/lib/auth.ts:30` e desligar a flag no servidor, dentro do hook `after` de `changePassword` do better-auth. Acrescentar a verificação da flag em `requireMemberContext` para que as Server Actions também a respeitem. cobre F3
4. Adicionar guarda de ambiente no topo de `scripts/seed.ts` (abortar se `NODE_ENV === "production"` ou se a `DATABASE_URL` não for local, salvo `ALLOW_SEED=1` explícito) e excluir `scripts/seed.ts` e `scripts/screenshots.mjs` do `.dockerignore`. cobre F7

P3 Endurecimento

1. Substituir `'unsafe-inline'` de `script-src` por CSP com nonce gerado em `proxy.ts` — caminho já apontado no comentário de `next.config.ts:3-7`. cobre F12
2. Validar a força de `FLOW_SECRETS_KEY` na inicialização (rejeitar valores de baixa entropia) e acrescentar validação própria de `BETTER_AUTH_SECRET` que recuse placeholders conhecidos, sem depender só do que a biblioteca checa. cobre F9, F10
3. Alinhar o matcher de `src/proxy.ts:30-43` às rotas do grupo (`app`), ou trocá-lo por um padrão negativo que cubra tudo exceto assets — e registrar no arquivo que o matcher é otimização, não proteção. cobre F6
4. Acrescentar `*.xlsx`, `*.xls` e `*.csv` ao `.gitignore` e mover a planilha do escritório para fora da árvore do repositório. cobre F11
5. Ou consumir `escapeTelegramHtml` de fato (ligando `parse_mode: HTML` com escape em todo texto de usuário), ou remover a função e o tipo `parseMode` para que a armadilha deixe de existir. cobre F13

Issues para o GitHub

Texto completo, em Markdown, pronto para copiar e colar. Achados relacionados foram agrupados numa issue só quando a correção é a mesma. As marcadas **Resolvida** já foram corrigidas e verificadas nesta revisão — ficam aqui como registro; as demais seguem acionáveis.

Issue 1

RESOLVIDA

security

severity:alta

backend

achados: F1, F2

--- ISSUE 1 ---

[Segurança] Server Actions de Fechamentos aceitam qualquer membro da organização

Labels: security, severity:alta, backend

Problema

As 7 Server Actions de `src/app/(app)/clans/[id]/closing-actions.ts` verificam apenas que quem chamou é membro da organização (`requireMemberContext()`). Nenhuma delas checa clã ou papel.

A restrição "Fechamentos pertence à Contabilidade" existe só na navegação:

- `src/lib/clan-tabs.ts:37` só oferece a aba `closings` ao clã de slug `contabilidade`;
- `src/app/(app)/clans/[id]/page.tsx:133` devolve 404 para quem não é do clã.

Nenhuma dessas barreiras existe no caminho da Server Action. Como `closing-board.tsx` é um Client Component que importa as actions (`:44-47`), os identificadores `Next-Action` estão em um chunk estático público sob `\_next/static/chunks/`. Qualquer usuário autenticado consegue endereçá-las diretamente.

O agravante está em `setYearClosed`: ela credita XP **ao próprio chamador**, aceitando qualquer `clientId` da organização e qualquer `year` entre 2000 e 2100. A idempotência é por par (empresa, ano), não por pessoa — com ~250 empresas e 101 anos, um `member` gera mais de 25 mil créditos legítimos aos olhos do banco. Como o `xp\_ledger` é imutável por privilégio (REVOKE UPDATE, DELETE), os lançamentos falsos só podem ser estornados um a um.

O contraste interno mostra que é lapso, não decisão: o arquivo irmão da mesma aba, `commitment-actions.ts`, protege as suas 8 actions com `requireDistributionManager` (`:45-67`).

Evidência

`src/app/(app)/clans/[id]/closing-actions.ts:281` - `deleteClosing`, controle de acesso na íntegra:

```
``ts
export async function deleteClosing(
  input: z.input<typeof deleteClosingSchema>,
): Promise<ActionResult> {
  const ctx = await requireMemberContext(); // só prova que é membro da org
  if (!ctx.ok) return ctx;

  const parsed = deleteClosingSchema.safeParse(input);
  if (!parsed.success) return err("Fechamento inválido.");

  const deleted = await withOrgTx(ctx.orgId, (tx) =>
    tx.delete(schema.accountingClosings).where(
      and(
        eq(schema.accountingClosings.id, parsed.data.closingId),
        eq(schema.accountingClosings.orgId, ctx.orgId),
      ),
    ).returning({ id: schema.accountingClosings.id }),
  );
}
```

`src/app/(app)/clans/[id]/closing-actions.ts:384-396` - `setYearClosed`, crédito de XP:

```
``ts
let xpAwarded = false;
if (data.closed) {
  const credited = await tx
    .insert(schema.xpLedger)
    .values({
```

```

    orgId: ctx.orgId,
    userId: ctx.userId,          // o próprio chamador recebe o XP
    closingYearId: annual.id,
    amount: CLOSING_YEAR_XP,
    reason: "closing_year_closed",
  })
  .onConflictDoNothing()
  .returning({ id: schema.xpLedger.id });
  xpAwarded = credited.length > 0;
}
```

```

Actions afetadas: `createClosing` (:88), `updateClosing` (:146), `setClosingStatus` (:214), `deleteClosing` (:281), `setYearClosed` (:316), `setDefisCompleted` (:440), `updateYearNotes` (:506).

#### ## Impacto

- Qualquer membro da organização apaga, cria e reescreve fechamentos contábeis de qualquer empresa-cliente – incluindo saldo de caixa, resultado, empréstimo de sócio e observações.
- Farm de XP em massa, destruindo o ranking e o sistema de níveis.
- Fechar anos arbitrários dispara notificação para a organização inteira (:400-419) e bloqueia o registro da DEFIS (:472).

#### ## Correção sugerida

1. Criar `requireClosingManager(tx, ctx, clanId)` em `closing-actions.ts`, espelhando `requireDistributionManager` (`commitment-actions.ts:45-67`): carregar os fatos com `loadClanScopedFacts`, exigir `clan.slug === CONTABILIDADE\_CLAN\_SLUG` e delegar a decisão a uma nova régua pura `canManageClanClosings` em `src/domain/guild-permissions.ts`.
2. Acrescentar `clanId: z.uuid()` aos schemas Zod das 7 actions e passar o `clanId` do call site (`closing-board.tsx` já recebe `clanId` via `ClosingsTab`).
3. Auditar o `xp\_ledger` procurando lançamentos `closing\_year\_closed` sem fechamento legítimo e estornar com `reason = 'reversal'` – nunca DELETE, que o role da aplicação não tem.

#### ## Critérios de aceite

- [ ] `canManageClanClosings` existe em `src/domain/guild-permissions.ts` com teste unitário cobrindo member, líder da Contabilidade, líder de outro clã, admin e owner.
- [ ] As 7 actions de `closing-actions.ts` chamam `requireClosingManager` antes de qualquer leitura ou escrita.
- [ ] Chamar qualquer das 7 actions como `member` sem vínculo com a Contabilidade devolve erro de autorização e não altera nenhuma linha.
- [ ] Chamar `setYearClosed` com um `clanId` de clã que não seja a Contabilidade devolve erro e não insere no `xp\_ledger`.
- [ ] Líder da Contabilidade e admin/owner continuam executando as 7 actions com sucesso (teste de não-regressão).
- [ ] Lançamentos indevidos de `closing\_year\_closed` existentes no banco foram estornados e o ranking foi conferido.

--- FIM ISSUE 1 ---

## Issue 2

RESOLVIDA

security

severity:média

auth

achados: F3

--- ISSUE 2 ---

# [Segurança] mustChangePassword é desligado pelo navegador sem trocar a senha

\*\*Labels:\*\* security, severity:média, auth

#### ## Problema

O campo `mustChangePassword` é declarado com `input: true` em `src/lib/auth.ts:25-33`, o que faz o better-auth aceitá-lo no corpo do endpoint `/api/auth/update-user` – confirmado em `node\_modules/better-auth/dist/api/routes/update-user.mjs:54`, `parseUserInput(ctx.context.options, rest, "update")`.

Quem desliga a flag hoje é o navegador, numa segunda chamada logo após a troca de senha (`change-password-form.tsx:54`). Nada no servidor amarra o desligamento da flag ao fato de a senha ter sido efetivamente trocada. Qualquer usuário autenticado pode emitir `POST /api/auth/update-user {"mustChangePassword": false}` e seguir operando com a senha temporária que o admin distribuiu por canal externo.

Como agravante, `requireMemberContext` (`src/lib/action-context.ts:23-49`) não olha a flag – então mesmo sem desligá-la a pessoa já executa qualquer Server Action. A barreira do `/change-password` existe só na navegação de

páginas (`src/lib/session.ts:33-35`).

## ## Evidência

`src/lib/auth.ts:25-33`:

```
```ts
user: {
  additionalFields: {
    mustChangePassword: {
      type: "boolean",
      defaultValue: false,
      input: true,          // aceito no corpo de /update-user
    },
  },
},
},
```
```

`src/app/(auth)/change-password/change-password-form.tsx:53-54`:

```
```ts
// Marca a troca como feita - deixa de ser redirecionado para cá.
await authClient.updateUser({ mustChangePassword: false });
```
```

`src/lib/session.ts:31-41` (única aplicação da flag, e só em páginas):

```
```ts
export async function requireOrgSession() {
  const session = await requireSession();
  if (session.user.mustChangePassword) {
    redirect("/change-password");
  }
}
```
```

## ## Impacto

Senhas temporárias distribuídas fora do sistema (chat, e-mail, papel) permanecem válidas por tempo indeterminado. A janela de tomada de conta se estende indefinidamente - inclusive para contas `admin` recém-criadas por `createMemberWithTempPassword`.

## ## Correção sugerida

1. Remover `input: true` de `mustChangePassword` em `src/lib/auth.ts:30`.
2. Desligar a flag no servidor, como efeito da troca de senha - hook `after` de `changePassword` nos `databaseHooks`, ou uma Server Action dedicada que faça as duas coisas na mesma transação.
3. Remover a chamada `authClient.updateUser({ mustChangePassword: false })` de `change-password-form.tsx:54`.
4. Acrescentar a verificação da flag em `requireMemberContext`, para que as Server Actions respeitem a mesma regra das páginas.

## ## Critérios de aceite

- [ ] `POST /api/auth/update-user` com `{ "mustChangePassword": false }` não altera a flag (campo rejeitado ou ignorado).
- [ ] Trocar a senha por `authClient.changePassword` desliga a flag sem nenhuma chamada adicional do cliente.
- [ ] Usuário com `mustChangePassword = true` recebe erro de autorização ao chamar qualquer Server Action, e não só ao navegar.
- [ ] O fluxo completo (admin cria membro com senha temporária → membro entra → troca a senha → acessa o dashboard) continua funcionando de ponta a ponta.

--- FIM ISSUE 2 ---

### Issue 3

RESOLVIDA

security

severity:média

database

achados: F4, F5

--- ISSUE 3 ---

# [Segurança] 12 tabelas de domínio sem FORCE ROW LEVEL SECURITY, e o check:rls não as cobre

\*\*Labels:\*\* security, severity:média, database

## ## Problema

Das 42 tabelas de domínio, 30 recebem `FORCE ROW LEVEL SECURITY` e 12 ficaram só com `ENABLE`. Sem `FORCE`, o \*\*dono da tabela ignora a política\*\* – o RLS dessas 12 só vale enquanto a aplicação conectar com um role que não seja o `postgres`.

Tabelas sem `FORCE`: `tasks`, `task\_events`, `xp\_ledger`, `clients`, `accounting\_closings`, `accounting\_closing\_years`, `mission\_templates`, `mission\_template\_items`, `telegram\_connections`, `telegram\_link\_tokens`, `telegram\_preferences`, `telegram\_outbox`. São, justamente, as do núcleo do produto.

Em operação normal não há furo: `src/db/index.ts:6-15` e `docker-compose.yml:43` conectam com `guilda\_app`. Mas a própria base antecipa o cenário contrário – `scripts/start-production.mjs:22-28` avisa em runtime que, sem `MIGRATION\_DATABASE\_URL`, as migrations rodam com `DATABASE\_URL`, e `docker-compose.yml:5-6` documenta o deploy com Postgres externo, em que apontar as duas variáveis para o mesmo role é o erro natural. Nessa configuração o isolamento continuaria valendo nas 30 tabelas com `FORCE` e sumiria silenciosamente nas 12 do núcleo.

O motivo de isso ter passado por 60 migrations é o segundo problema: `scripts/check-rls.mjs:13-38` só audita as 24 tabelas "novas" – a suíte que existia para pegar essa regressão não olha para o núcleo.

## ## Evidência

`src/db/migrations/0002\_rls-domain.sql:11-17`:

```
```sql
ALTER TABLE "tasks" ENABLE ROW LEVEL SECURITY;
CREATE POLICY "org_isolation" ON "tasks"
  USING ("org_id" = current_setting('app.org_id', true));

ALTER TABLE "task_events" ENABLE ROW LEVEL SECURITY;
CREATE POLICY "org_isolation" ON "task_events"
  USING ("org_id" = current_setting('app.org_id', true));
```
```

O padrão adotado a partir de 2026 – `src/db/migrations/0025\_force-rls-informatives.sql:7`:

```
```sql
-- Sem FORCE, o OWNER da tabela ignora a politica; a aplicacao conecta como
-- guilda_app (nao-owner) e ja era isolada, entao isto e defesa em profundidade.
ALTER TABLE "informatives" FORCE ROW LEVEL SECURITY;
```
```

`scripts/check-rls.mjs:13-38` – a lista auditada, sem as tabelas do núcleo:

```
```js
const NEW_TABLES = [
  "clan_informative_routes", "task_assignee_suggestions", "guild_notices",
  "guild_notice_reads", "informatives", "fiscal_portfolios", /* ... */
  "company_flows", "company_flow_secrets", "company_flow_events",
];
```
```

Outras migrations afetadas: `0004\_rls-xp-ledger.sql:3`, `0007\_rls-clients.sql:4`, `0009\_rls-mission-templates.sql`, `0011\_rls-accounting-closings.sql:2`, `0017\_mute-red-wolf.sql`.

## ## Impacto

Sob uma configuração de banco plausível e não bloqueada por nada, a segunda camada de defesa contra vazamento entre organizações some das quatro tabelas mais sensíveis (missões, ledger de XP, empresas-cliente e fechamentos), restando apenas o filtro por `org\_id` na aplicação.

## ## Correção sugerida

1. Nova migration aplicando `FORCE ROW LEVEL SECURITY` às 12 tabelas, seguindo o padrão de `0025\_force-rls-informatives.sql`. Atenção ao precedente de `0041\_fiscal\_unknown\_backfill.sql:14-15,65-66`: migrations de backfill que rodam como owner precisam de `NO FORCE` temporário.
2. Trocar a lista fixa `NEW\_TABLES` de `check-rls.mjs` por uma consulta a `pg\_class`/`information\_schema.columns` que exija `relrowsecurity AND relforcerowsecurity` em \*\*todas\*\* tabela que tenha coluna `org\_id` – assim novas tabelas passam a ser cobertas sozinhas.

## ## Critérios de aceite

```
- [] `SELECT relname FROM pg_class WHERE relrowsecurity AND NOT relforcerowsecurity` não devolve nenhuma tabela com coluna `org_id`.
- [] `npm run check:rls` deriva a lista do banco em vez de uma constante e falha se qualquer tabela com `org_id` estiver sem `ENABLE` ou sem `FORCE`.
- [] `npm run check:rls` passa inteiro após a migration.
- [] Seed e migrations continuam rodando (nenhuma quebra por `FORCE` aplicado ao role dono).

--- FIM ISSUE 3 ---
```

**Issue 4****RESOLVIDA**

security

severity:média

devops

achados: F7

--- ISSUE 4 ---

# [Segurança] Seed cria owner com senha fixa e viaja dentro da imagem de produção

\*\*Labels:\*\* security, severity:média, devops

## ## Problema

`scripts/seed.ts` cria uma conta com papel `owner` usando a senha fixa `demo123456`, documentada publicamente no `README.md:113` e repetida em `scripts/screenshots.mjs:17`.

Não existe guarda de ambiente: a única condição de parada é a organização `guilda-demo` já existir (`:87-94`), o que não impede a primeira execução contra um banco de produção.

E o script está dentro da imagem publicada. O `.dockerignore:9` exclui `scripts/e2e-\*.mjs` mas não `scripts/seed.ts`, e o `Dockerfile:42` copia o diretório inteiro para o estágio `runner`. A imagem final também tem `tsx` em `node\_modules` e o alvo `npm run seed` no `package.json:16` – o caminho está pronto.

## ## Evidência

`scripts/seed.ts:7,21,24-29,78`:

```
``ts
* Login demo: helena@demo.guilda.dev / demo123456 (e demais e-mails)
const PASSWORD = "demo123456";
const PEOPLE = [
 { key: "helena", name: "Helena Prado", email: "helena@demo.guilda.dev", role: "owner" },
 // ...
];
await auth.api.signUpEmail({ body: { name, email, password: PASSWORD } });
``
```

`Dockerfile:42`:

```
``dockerfile
COPY --from=build --chown=nextjs:nodejs /app/scripts ./scripts
``
```

`scripts/screenshots.mjs:17`:

```
``js
await page.fill("#password", "demo123456");
``
```

## ## Impacto

Uma execução acidental de `npm run seed` contra produção – por `docker exec`, por erro de operação na VPS, ou por um `DATABASE\_URL` apontado para o lugar errado – cria uma conta `owner` com credencial de conhecimento público em uma organização real. Um `owner` tem controle total do tenant, inclusive leitura das senhas Gov.br descriptografadas via `revealCompanyFlowGovPassword`.

## ## Correção sugerida

1. Guarda no topo de `scripts/seed.ts`: abortar se `process.env.NODE\_ENV === "production"` ou se a `DATABASE\_URL` não apontar para `localhost`/`127.0.0.1`, salvo `ALLOW\_SEED=1` explícito.
2. Acrescentar `scripts/seed.ts` e `scripts/screenshots.mjs` ao `.dockerignore` – ou restringir o `COPY` do `Dockerfile:42` aos scripts que a produção realmente usa (`start-production.mjs`, `telegram-worker.ts`, `load-env.ts`).
3. Ler a senha da demo de `process.env.SEED\_PASSWORD` com fallback só em ambiente local, e ajustar o

`README.md:113` de acordo.

## ## Critérios de aceite

- [ ] `NODE\_ENV=production npm run seed` aborta com mensagem clara e sem tocar o banco.
- [ ] `docker run --rm --entrypoint ls <imagem> scripts` não lista `seed.ts` nem `screenshots.mjs`.
- [ ] `npm run seed` continua funcionando no ambiente local documentado no README.
- [ ] O README descreve como definir a senha da demo em vez de publicá-la fixa.

--- FIM ISSUE 4 ---

## Issue 5

RESOLVIDA

security

severity:baixa

devops

achados: F8, F9, F10

--- ISSUE 5 ---

# [Segurança] Higiene de segredos: credenciais fixas de dev, placeholder no Dockerfile e falta de validação de inicialização

\*\*Labels:\*\* security, severity:baixa, devops

## ## Problema

Três itens do mesmo tema – segredos que existem como literais ou que entram sem aferição – agrupados numa issue só porque a correção é a mesma rotina de endurecimento de configuração.

\*\*(a) Credenciais de desenvolvimento fixas.\*\* `docker/dev/init/01-roles.sql:5` cria o role com senha literal `guilda\_app\_dev`; `.env.example:3,5` repete `guilda\_app\_dev` e `postgres:postgres`; `docker-compose.dev.yml:9,11-12` usa `POSTGRES\_PASSWORD: postgres` e publica a porta 5432 no host sem restringir a interface. O equivalente de produção está correto (`docker/prod/init/01-roles.sh:7` interpola `\${APP\_DB\_PASSWORD}` e `docker-compose.yml:15` exige a variável com `:?`), então o risco é local.

\*\*(b) Placeholder de segredo no Dockerfile.\*\* `Dockerfile:17` define `ENV BETTER\_AUTH\_SECRET=placeholder-somente-para-o-build-32ch` no estágio `build`. Verificado: o estágio `runner` parte de `FROM base` (`:32`) e \*\*não\*\* herda esse `ENV` – o valor não vira o segredo de runtime. O que resta é que o literal tem 36 caracteres e não é o `DEFAULT\_SECRET` do better-auth, logo passaria pelas duas travas da biblioteca (`node\_modules/better-auth/dist/context/create-context.mjs:41-44`) se um dia fosse promovido a runtime por engano.

\*\*(c) Sem validação própria de inicialização.\*\* O projeto não checa a força de `BETTER\_AUTH\_SECRET` nem de `FLOW\_SECRETS\_KEY`. `src/lib/company-flows/secrets.ts:13-20` aceita qualquer valor que decodifique para 32 bytes – e essa chave protege senhas Gov.br de empresas-cliente. Como o `Dockerfile:49-50` registra que o painel de hospedagem constrói só o Dockerfile, sem o Compose, o `\${VAR:?...}` – que é a trava real do projeto – não roda nesse caminho de implantação.

## ## Evidência

`docker/dev/init/01-roles.sql:5`:

```
```sql
CREATE ROLE guilda_app LOGIN PASSWORD 'guilda_app_dev' NOSUPERUSER NOCREATEDB NOCREATEROLE;
```
```

`Dockerfile:14-18`:

```
```dockerfile
# Placeholders APENAS para o build (todas as páginas são dinâmicas –
# nenhuma conexão acontece). Os valores reais entram em runtime.
ENV DATABASE_URL=postgresql://build:build@localhost:5432/build
ENV BETTER_AUTH_SECRET=placeholder-somente-para-o-build-32ch
ENV BETTER_AUTH_URL=http://localhost:4000
```
```

`src/lib/company-flows/secrets.ts:13-20`:

```
```ts
function encryptionKeyFromEnvironment(): Buffer | null {
  const value = process.env.FLOW_SECRETS_KEY?.trim();
  if (!value) return null;
  const key = /^[A-Za-f0-9]{64}$/.test(value)
    ? Buffer.from(value, "hex")
    : Buffer.from(value, "base64");
}
```

```

    return key.length === 32 ? key : null; // só o tamanho é conferido
  }
  ```

 ## Impacto

 - (a) Acesso ao banco de desenvolvimento por quem estiver na mesma rede. Sem impacto em produção.
 - (b) Nenhum vazamento hoje – é a ausência de uma trava caso o placeholder seja promovido.
 - (c) Chave fraca configurada por engano protege senhas de portal de governo sem que nada avise.

 ## Correção sugerida

 1. Criar `src/lib/env-guard.ts` importado no boot da aplicação, que valide `BETTER_AUTH_SECRET` (≥ 32 caracteres, entropia mínima, recusa a uma lista de placeholders conhecidos incluindo o do Dockerfile) e `FLOW_SECRETS_KEY` (entropia mínima), lançando em produção.
 2. Trocar o placeholder do `Dockerfile:17` por um valor gerado no build (`RUN BETTER_AUTH_SECRET=$(openssl rand -base64 32) npm run build`) ou por um sentinela explicitamente na lista de recusa do guard.
 3. Ligar a porta do Postgres de dev a `127.0.0.1:5432:5432` em `docker-compose.dev.yml:12` e trocar a senha literal de `docker/dev/init/01-roles.sql:5` por `${APP_DB_PASSWORD:-guilda_app_dev}`, alinhando ao script de produção.

 ## Critérios de aceite

 - [] Subir a aplicação em produção com `BETTER_AUTH_SECRET=placeholder-somente-para-o-build-32ch` falha na inicialização com mensagem clara.
 - [] Subir com `FLOW_SECRETS_KEY` de baixa entropia falha na inicialização.
 - [] O Postgres de desenvolvimento não aceita conexão de outra máquina da rede.
 - [] `docker compose -f docker-compose.dev.yml up -d` seguido de `npm run dev` continua funcionando com o `.env.example` copiado.

 --- FIM ISSUE 5 ---

```

## Issue 6

RESOLVIDA

security

severity:baixa

frontend

achados: F12

```

--- ISSUE 6 ---

[Segurança] CSP ainda depende de 'unsafe-inline' em script-src

Labels: security, severity:baixa, frontend

Problema

`next.config.ts:10` define `script-src 'self' 'unsafe-inline'`. O comentário logo acima (`:3-7`) já reconhece a limitação e aponta o caminho: nonce via proxy.

Com `unsafe-inline`, a CSP deixa de funcionar como segunda linha de defesa contra XSS. **Nenhum sink de XSS foi encontrado no projeto** – busca por `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function`, `srcDoc` e `javascript:` em todo `src/` deu zero ocorrências –, então isto é endurecimento, não correção de vulnerabilidade explorável.

Evidência

`next.config.ts:8-20`:

```ts
const contentSecurityPolicy = [
  "default-src 'self'",
  "script-src 'self' 'unsafe-inline'", // anula boa parte da CSP
  "style-src 'self' 'unsafe-inline'",
  "img-src 'self' data: blob:",
  "font-src 'self'",
  "connect-src 'self'",
  "object-src 'none'",
  "base-uri 'self'",
  "form-action 'self'",
  "frame-ancestors 'none'",
  "upgrade-insecure-requests",
].join("; ");
```

O que já está correto e deve permanecer: `object-src 'none'`, `base-uri 'self'`, `form-action 'self'`, `frame-

```



ancestors 'none' e a ausência de 'unsafe-eval'.

## ## Impacto

Perda da mitigação em profundidade contra XSS. Sem impacto isolado, já que não existe sink no código atual – mas se um for introduzido, o cabeçalho não segura.

## ## Correção sugerida

1. Gerar um nonce por requisição em 'src/proxy.ts', propagá-lo pelo header 'x-nonce' e montar a CSP dinamicamente com 'script-src 'self' 'nonce-<valor>' 'strict-dynamic'.
2. Mover os cabeçalhos de 'next.config.ts:47-54' para o proxy (a CSP passa a ser por requisição; os demais podem ficar estáticos).
3. Manter 'style-src 'unsafe-inline'' se necessário – o Tailwind e o Next injetam estilo inline, e o risco de 'style-src' é bem menor que o de 'script-src'.

## ## Critérios de aceite

- [ ] O header 'Content-Security-Policy' das respostas HTML contém 'nonce-' e não contém 'unsafe-inline' em 'script-src'.
- [ ] O nonce muda a cada requisição.
- [ ] Nenhum erro de CSP no console em '/dashboard', '/tasks', '/tasks/new', '/clans/[id]' (todas as abas), '/profile' e '/sign-in'.
- [ ] As demais diretivas ('object-src 'none'', 'base-uri', 'form-action', 'frame-ancestors') permanecem inalteradas.

--- FIM ISSUE 6 ---

## Issue 7

RESOLVIDA

security

severity:informativa

manutenção

achados: F6, F11, F13

--- ISSUE 7 ---

# [Segurança] Higiene de configuração: matcher do proxy, planilha fora do .gitignore e escape do Telegram sem consumidor

\*\*Labels:\*\* security, severity:informativa, manutenção

## ## Problema

Três itens sem impacto explorável hoje, agrupados porque cada um é uma armadilha esperando alguém no futuro – e a correção de todos cabe num único PR.

**(a) Matcher do proxy desalinhado.** 'src/proxy.ts:30-43' lista '/dashboard', '/tasks', '/clients', '/campaigns', '/leaderboard', '/members', '/profile', '/onboarding', '/change-password', '/sign-in' e '/sign-up', mas não '/clans', '/settings', '/informativos', '/mural' nem '/closings'. **\*\*Não abre acesso\*\***: 'src/app/(app)/layout.tsx:13' chama 'requireOrgSession()', que valida a sessão de verdade, e o próprio 'proxy.ts:5-8' documenta que a checagem dele é otimista. O risco é alguém no futuro presumir que "o matcher é a proteção" e criar uma rota que dependa disso.

**(b) Planilha na raiz sem regra no .gitignore.** 'Planilha contole de Notas do escritório.xlsx' está não rastreada na raiz do repositório, e o '.gitignore' não tem nenhuma regra para planilhas. O histórico do Git está limpo – a varredura não achou '.env', chave privada nem token em nenhum commit –, mas um 'git add .' inclui o arquivo, e o nome sugere dados fiscais reais de empresas-cliente.

**(c) 'escapeTelegramHtml' sem consumidor.** 'src/lib/telegram/format.ts:3-9' define a função, 'src/lib/telegram/endpoint.ts:58' define o tipo 'parseMode?: "HTML"' e ':115' só envia 'parse\_mode' se alguém pedir. Nenhum chamador de produção define 'parseMode', então as mensagens saem como texto puro e não há injeção hoje. Títulos de missão e notas de rejeição entram sem escape ('tasks/actions.ts:443,460,472'), o que está correto para texto puro – e vira injeção no instante em que alguém ligar o 'parse\_mode' achando que o escape já é aplicado.

## ## Evidência

'src/proxy.ts:30-43':

```ts

```
export const config = {
  matcher: [
    "/dashboard/:path*", "/tasks/:path*", "/clients/:path*",
    "/campaigns/:path*", "/leaderboard/:path*", "/members/:path*",
    "/profile/:path*", "/onboarding", "/change-password",
```



```

    "/sign-in", "/sign-up",
  ],
};
```

`git status --short`:

```
?? .impeccable/
?? "Planilha contole de Notas do escritório.xlsx"
```

`src/lib/telegram/format.ts:3-9` (nenhum chamador em produção):

```ts
/** Escapa texto não confiável para `parse_mode: HTML` do Telegram. */
export function escapeTelegramHtml(value: string): string {
  return value.replaceAll("&", "&amp;").replaceAll("<", "&lt;").replaceAll(">", "&gt;");
}
```

Impacto

Nenhum hoje, nos três casos. São riscos de manutenção: uma rota futura que confie no matcher; um `git add .` que comite dados de clientes de forma irreversível no histórico; um `parse_mode: HTML` ligado sem escape.

Correção sugerida

1. Substituir o matcher de `src/proxy.ts:30-43` por um padrão negativo `"/((?!api|_next/static|_next/image|favicon.ico).*)"`) e reforçar no comentário que ele é otimização e não proteção.
2. Acrescentar `*.xlsx`, `*.xls` e `*.csv` ao `.gitignore` e mover a planilha para fora da árvore do repositório.
3. Decidir sobre o Telegram: ou passar a usar `parse_mode: "HTML"` aplicando `escapeTelegramHtml` a todo texto de origem de usuário, ou remover a função e o tipo `parseMode` para que a armadilha deixe de existir.

Critérios de aceite

- [] Um visitante não autenticado é redirecionado para `/sign-in` em `/clans`, `/settings`, `/informativos`, `/mural` e `/closings` sem que o layout do app seja renderizado antes.
- [] `git status --short` não lista nenhum arquivo de planilha, e `git check-ignore -v "Planilha contole de Notas do escritório.xlsx"` confirma a regra.
- [] Ou `escapeTelegramHtml` tem ao menos um chamador de produção com teste cobrindo texto contendo `<`, `>` e `&`, ou a função e o tipo `parseMode` foram removidos e `npm run test` passa.

--- FIM ISSUE 7 ---

```